

Cross-Cultural Dynamics of Emotional Expression in Lyrics

Process Book

Team InSight: Weilun Xu, Xin Huang, Qi Wang

Website: <https://com-480-data-visualization.github.io/lyrical-emotion-InSight/>

1. Introduction and Initial Plan

Our project explores the fascinating world of cross-cultural emotional expression in song lyrics. We chose this topic because music is a universal language that transcends cultural boundaries, yet how emotions are expressed through lyrics can vary dramatically across different languages and cultural contexts.

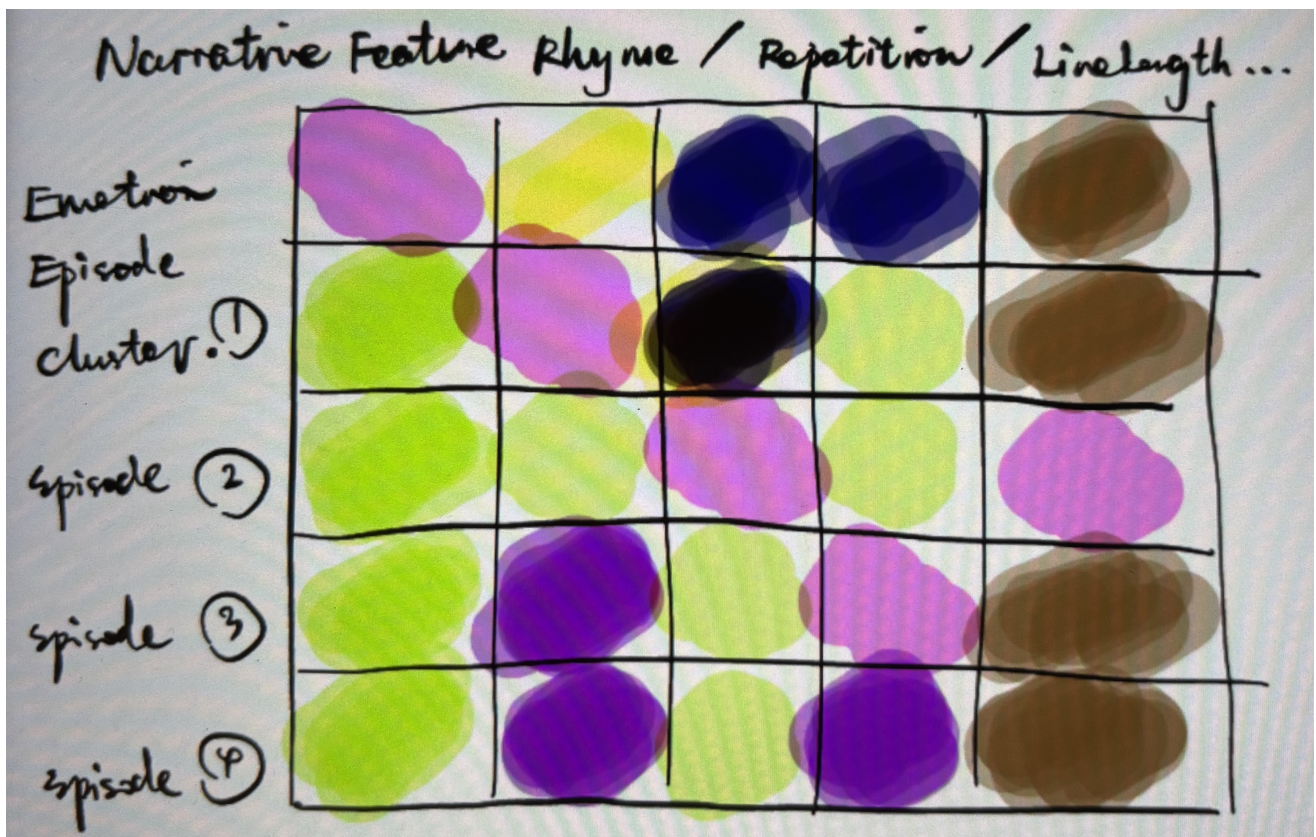


Figure: Initial conceptual sketch showing the relationship between lyrical structure and emotional expression

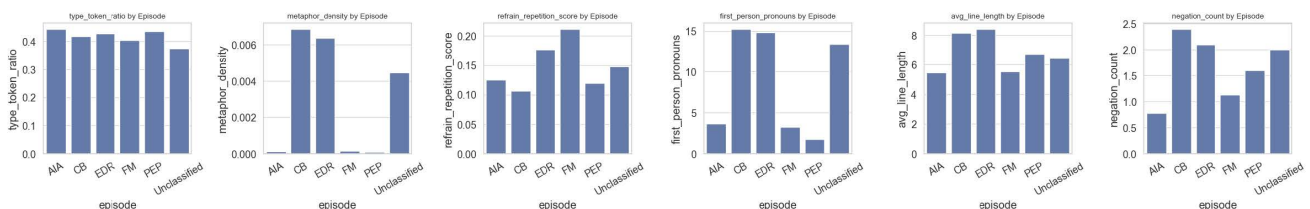


Figure: Linking Narrative Structure to Emotional Episode Trajectories

Our initial vision, as outlined in our Milestone 1 proposal, was to visualize:

1. Cross-cultural differences in emotional expression
2. Temporal trends in emotional tone
3. Correlation between linguistic structures and emotional content
4. The distribution of emotions in the valence-arousal space across languages

We planned to use the WASABI dataset, which contains metadata for over 2 million commercially released songs, including language, emotional dimensions, and lyrical content. This extensive dataset offered an opportunity to analyze emotional patterns across different languages and time periods.

2. Data Processing Journey

Initial Data Exploration

Our first major challenge was the sheer size and complexity of the WASABI dataset. With over 2 million songs and dozens of features, we needed to identify which aspects were most relevant to our research questions.

Weilun performed extensive exploratory data analysis to understand the dataset's structure and potential insights. Initial findings revealed:

- Significant variations in valence and arousal scores across languages
- Temporal trends showing shifts in emotional expression over decades
- Rich metadata that could be leveraged for multi-dimensional analysis

Data Preparation Challenges

Converting the raw WASABI dataset into visualization-ready data required several processing steps:

1. **Missing Data:** Many songs lacked complete emotional annotations. We implemented a filtering process to ensure we only worked with songs that had reliable valence, arousal, and dominance (VAD) scores.
2. **Language Imbalance:** The dataset had significantly more English songs than other languages. To prevent bias, we used stratified sampling to ensure adequate representation of each language.
3. **Temporal Aggregation:** To visualize trends over time, we needed to aggregate emotional scores by year, decade, and language. This required careful statistical treatment to ensure accurate representation of trends.
4. **Derived Features:** We extracted and computed additional features from lyrics, including linguistic structure metrics (repetition, metaphor density, etc.) that were not directly available in the raw dataset.

Performance Optimization

The size of the dataset posed challenges for web-based visualization. To ensure smooth performance, we:

- Created smaller, purpose-built CSV / JSON files for each visualization module
- Pre-computed aggregations and statistics
- Implemented progressive loading techniques
- Used sampling techniques for scatter plots with large point counts

3. Design Evolution

From Sketches to Interactive Visualizations

Our design process evolved significantly from our initial sketches to the final implementation. We began with conceptual sketches and wireframes that outlined the key insights we wanted to convey.

A major design challenge was organizing multiple complex visualizations into a coherent narrative flow. We experimented with several approaches:

1. **Single-page scrolling design:** Initially, we placed all visualizations on a single page, but this became overwhelming and reduced the focus on individual insights.

2. **Dashboard approach:** We tried a dashboard layout with multiple small visualizations, but this limited the detail we could show in each chart.
3. **Final tab-based design:** We ultimately settled on a tab-based organization with related visualizations grouped together. This allowed for focused exploration while maintaining a cohesive narrative.

Visual Language Development

We developed a consistent visual language to unify the diverse visualizations:

- **Color scheme:** We created a carefully considered color palette that maintained consistency across modules while ensuring accessibility and clear differentiation between data categories.
- **Typography:** We selected Roboto for its readability and versatility across different visualization contexts.
- **Interactive elements:** We standardized hover states, tooltips, and transitions to create a coherent user experience.
- **Responsive design:** We implemented adaptive layouts to ensure visualizations remained effective across different screen sizes.

Addressing Feedback

After our first implementation, we gathered feedback which highlighted several issues:

1. **Cognitive overload:** Some visualizations contained too much information at once.
2. **Navigation confusion:** Users weren't clear on the relationship between visualizations.
3. **Performance issues:** Some animations were causing lag on slower devices.

Based on this feedback, we:

- Simplified several visualizations to focus on key insights
- Improved navigation with clearer section headings and descriptions
- Optimized performance by reducing animation complexity and implementing lazy loading

4. Implementation Challenges

Technical Complexity

Implementing our visualizations presented several technical challenges:

1. **Module Integration:** With 10+ visualization modules, we needed a robust framework to manage them. We initially created standalone HTML files for each visualization, then integrated them into a unified codebase. This presented challenges with shared resources and consistent styling.

```
// Example of our module initialization system
function initializeModulesForTab(tabId) {
  return new Promise((resolve, reject) => {
    try {
      // Clear any existing visualizations first
      cleanupVisualizationsForTab(tabId);

      if (tabId === 'emotion-across-culture') {
        initializeEmotionAcrossCultureModules().then(resolve).catch(reject);
      }
      else if (tabId === 'emotion-linguistics') {
        initializeEmotionLinguisticsModules().then(resolve).catch(reject);
      }
      else {

```

```

        resolve(); // Unknown tab, nothing to initialize
    }
} catch (error) {
    reject(error);
}
});
}

```

2. **Performance Optimization:** The UMAP visualization in particular required significant optimization. With thousands of data points, rendering was initially slow and caused browser lag. We implemented:
 - Canvas-based rendering for large point sets
 - Data sampling to reduce point counts while preserving patterns
 - Quadtree-based interaction for efficient point lookup on hover
3. **Responsive Design:** Creating visualizations that worked across device sizes required careful implementation:

```

function handleModuleEResize() {
    // Simply re-initialize the module which will recalculate all dimensions
    initModuleE();
}

```

4. **Cross-browser Compatibility:** Ensuring consistent behavior across browsers required extensive testing and fallback implementations.

D3.js Challenges

Working with D3.js presented specific challenges:

1. **Version Compatibility:** Our project used D3.js v7, which had significant API changes from previous versions. This required careful adaptation of examples and tutorials that used older versions.
2. **Complex Visualizations:** Some visualizations, like the Sankey diagram, required specialized D3 plugins and custom code to achieve the desired interactions.
3. **Animation Coordination:** Coordinating multiple animated elements to create cohesive transitions required careful timing and state management.
4. **Text Layout:** Ensuring readable text in charts (especially in the matrix visualizations) required careful positioning and styling.

5. Final Implementation

Our final implementation features a tabbed interface with three main sections:

1. **Overview:** Introduces the project and research questions
2. **Emotion Across Culture:** Visualizations exploring emotional dimensions across languages and time
3. **Emotion & Linguistics:** Visualizations of the relationship between linguistic features and emotional expression

Module Architecture

We implemented a modular code structure to improve maintainability:

```

project/
├─ index.html      # Main HTML file with page structure
├─ css/

```

```

|   ├── styles.css      # Main stylesheet
|   └── tabs.css        # Tab navigation styles
└── js/
    ├── common.js       # Shared utilities and functions
    ├── main.js         # Main initialization script
    ├── tabs.js         # Tab navigation functionality
    └── modules/        # Visualization modules
        ├── moduleA.js  # Historical Trends in Lyrical Tone
        ├── moduleB.js  # Temporal Emotion Trends by Language
        ├── moduleC.js  # Emotional Dimensions Across Languages
        ├── moduleD.js  # Clustering Analysis (Sankey Diagram)
        ├── moduleE.js  # Valence-Arousal Profiles
        ├── moduleEpisodeSign.js # Episode Signature Features
        ├── moduleEpisodeSig.js # Episode Signature Comparison
        ├── moduleMatrix.js # Confusion Matrix
        ├── moduleSpearman.js # Lyrical Feature Correlations
        ├── moduleStruct.js # Structural-Emotional Correlations
        ├── moduleUmap.js # UMAP Clustering Visualization
        └── moduleVAD.js  # VAD Correlation Matrix
└── data/               # Dataset files
    ├── processed_data/ # Language-specific data
    ├── processed_data2/ # Emotion dimension data
    └── various CSV/JSON files

```

Each visualization module follows a standard pattern:

1. Initialization function that sets up the visualization
2. Data loading and processing
3. SVG/Canvas creation and rendering
4. Interaction handlers
5. Resize handling for responsiveness

We implemented a shared utilities system for common functionality:

```

window.vizUtils = {
  tooltip: sharedTooltip,
  formatNumber: formatNumber,
  formatPercent: formatPercent,
  handleDataError: handleDataError,
  dataPath: dataPathHelpers,
  isElementInViewport: isElementInViewport
};

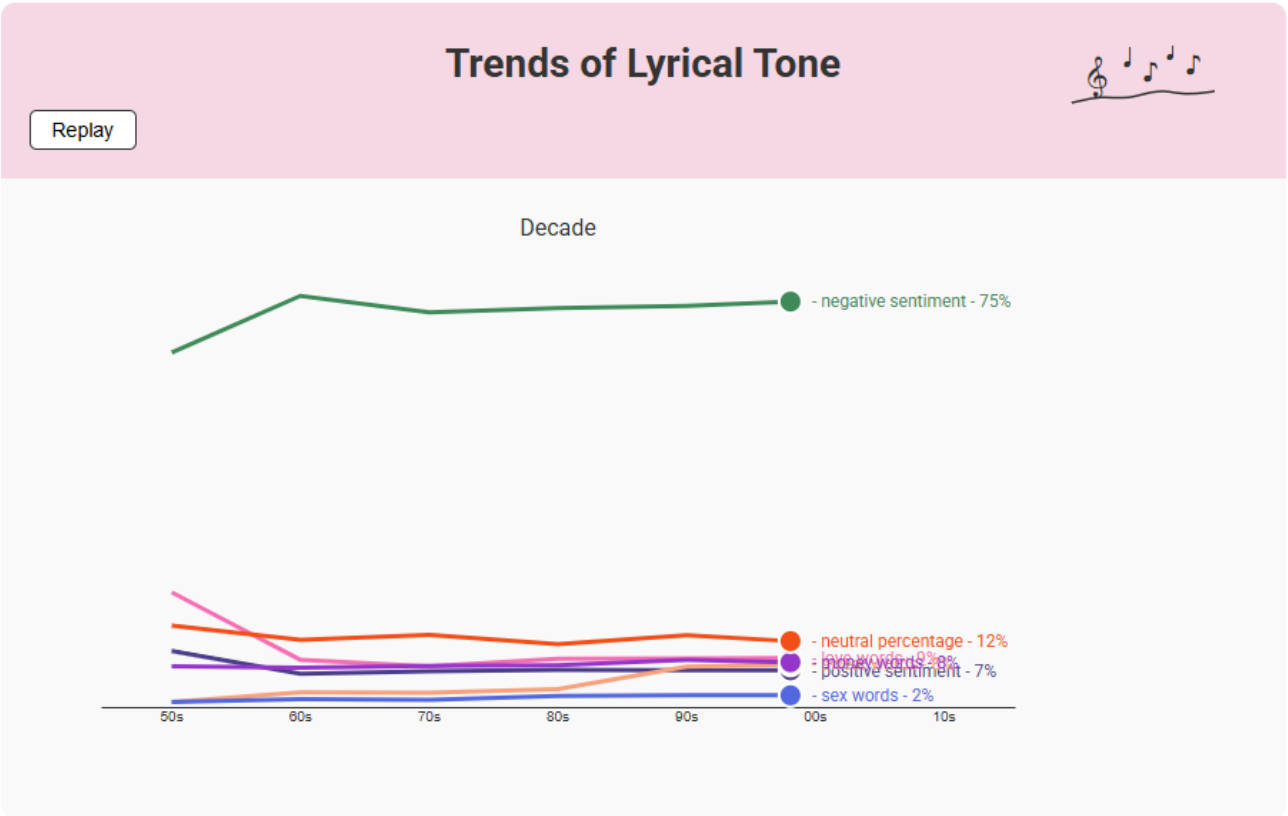
```

Key Visualizations

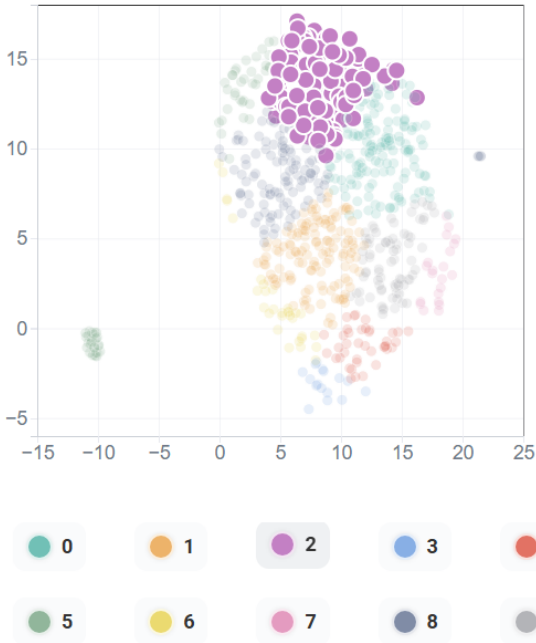
Our final implementation includes several innovative visualizations:

1. **Historical Trends in Lyrical Tone:** An animated line chart showing how emotional content in lyrics has changed over decades.
2. **Valence-Arousal Profiles:** A scatter plot positioning languages in a 2D emotional space, with an accompanying bar chart for detailed comparison.
3. **UMAP Clustering Comparison:** A dual visualization showing how structural patterns in lyrics map to emotional episodes.
4. **Confusion Matrix:** A logarithmic-scale heatmap showing the relationship between VAD values and emotional episode classifications.

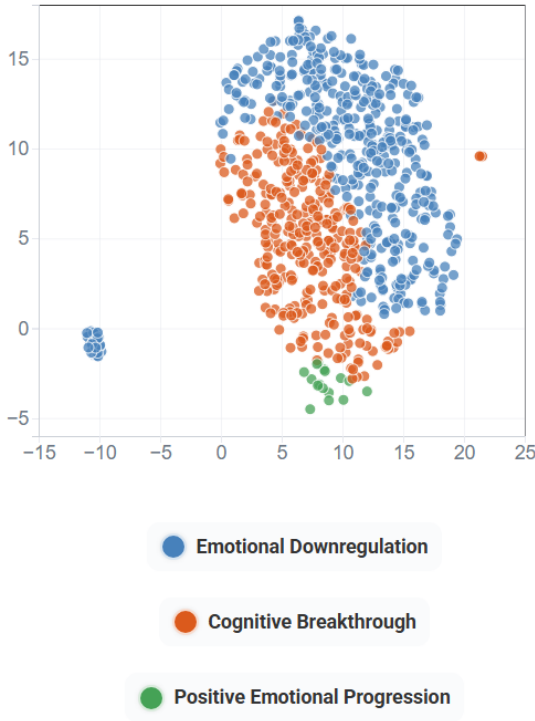
5. **Correlation Matrix:** A heatmap revealing relationships between structural features and emotional dimensions.

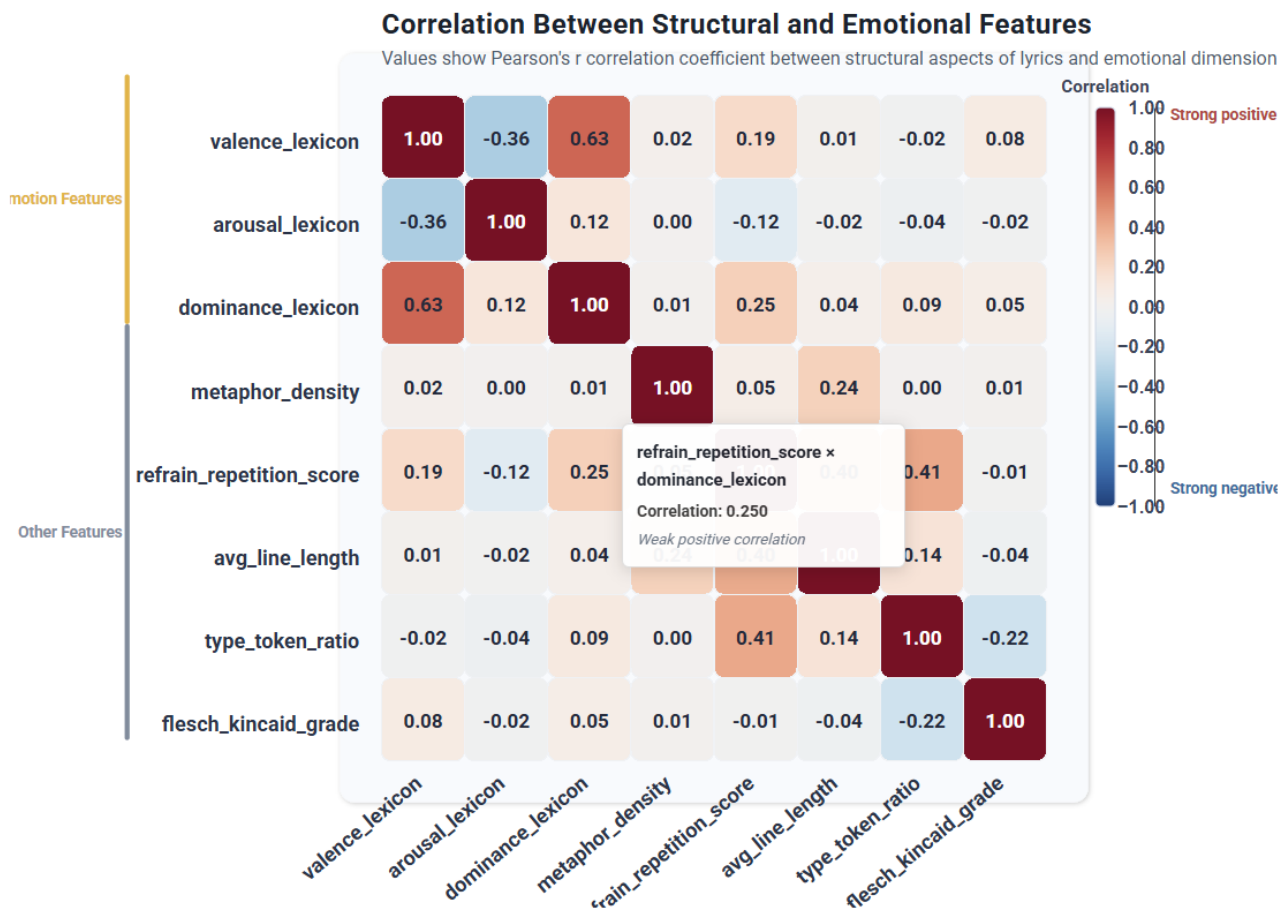


UMAP Projection Colored by KMeans Cluster



UMAP of Lyrics Colored by Dominant Episode





6. Lessons Learned

Throughout this project, we gained several valuable insights:

1. **Data Processing is Critical:** The quality of visualizations depends heavily on thoughtful data preparation. Investing time in creating clean, purpose-built datasets for each visualization paid off.
2. **Modular Architecture Benefits:** Our decision to create a modular code structure greatly facilitated collaboration and maintenance, despite the initial overhead.
3. **Performance Considerations:** Web-based visualizations require careful performance optimization, especially when dealing with large datasets or complex animations.
4. **Iterative Design Process:** Our approach of starting with simple implementations and progressively enhancing them helped us address unforeseen challenges.
5. **Narrative Structure Matters:** How visualizations are organized and presented significantly impacts the user's ability to extract insights.

7. Peer Assessment

The project was a collaborative effort with distinct contributions from each team member:

Weilun Xu:

- Led data analysis and established the project's narrative structure
- Conducted extensive exploratory data analysis
- Defined visualization requirements based on analytical insights
- Created textual content and explanations for the website
- Determined the logical flow and ordering of visualization modules
- Recorded the project screencast

Qi Wang:

- Handled the bulk of initial data processing for visualizations
- Extracted appropriate data subsets from the main dataset
- Created basic D3.js prototypes for each visualization concept
- Developed standalone HTML files for individual visualization modules
- Established foundational visualization structures

Xin Huang:

- Redesigned and rebuilt the initial visualizations with advanced animations and aesthetics
- Integrated all standalone visualizations into a cohesive web application
- Architected and Implemented the tab-based navigation system
- Created the responsive layout and cross-browser compatibility
- Optimized performance extensively through techniques including canvas rendering, data sampling, quadtree-based interactions, and progressive loading
- Developed the shared utilities and module management system
- Led the writing of the final process book and the GitHub Readme file

8. Conclusion

This project has allowed us to explore the fascinating intersection of data visualization, linguistics, and musicology. Through our interactive visualizations, we've created tools that allow users to explore how emotional expression in lyrics varies across languages, cultures, and time periods.

The modular architecture we developed not only facilitated the current implementation but also provides a foundation for future extensions. Potential future work could include:

1. Adding more languages and time periods as data becomes available
2. Incorporating audio features to explore the relationship between lyrical and musical emotional expression
3. Implementing more advanced linguistic analysis techniques
4. Adding comparative tools to explore specific language pairs in detail

Our process—from initial concept to final implementation—demonstrates the value of iterative design, collaborative development, and user-centered thinking in creating effective data visualizations.